CURSO DE GRADUAÇÃO EM COMPUTAÇÃO
DISCIPLINA DE PROGRAMAÇÃO ORIENTADA A OBJETOS — TURMA 0104NA

ARTHUR ROGERIO
CAMILE
DEBORA
VITOR CAUAN

SISTEMA ACADÊMICO E ASSOCIAÇÕES ENTRE CLASSES EM JAVA:

MODELAGEM COM CLASSE ASSOCIATIVA, ARQUITETURA EM CAMADAS E DESENVOLVIMENTO ORIENTADO A ESPECIFICAÇÕES

BRASIL
2026

ARTHUR ROGERIO
CAMILE
DEBORA
VITOR CAUAN

SISTEMA ACADÊMICO E ASSOCIAÇÕES ENTRE CLASSES EM JAVA:

MODELAGEM COM CLASSE ASSOCIATIVA, ARQUITETURA EM CAMADAS E DESENVOLVIMENTO ORIENTADO A ESPECIFICAÇÕES

Trabalho científico apresentado à disciplina de Programação Orientada a Objetos (Turma 0104NA), como requisito de avaliação teórica e prática sobre modelagem de associações entre classes, classe associativa, invariantes de consistência em memória, persistência em arquivos CSV e arquitetura em camadas utilizando Java puro (JDK 17+).

BRASIL
2026

RESUMO

A modelagem adequada de relacionamentos entre objetos constitui um dos pilares fundamentais da Engenharia de Software e da Programação Orientada a Objetos (POO), prevenindo anomalias de estado e acoplamento excessivo observados em soluções procedurais baseadas em classes monolíticas. Este trabalho apresenta a concepção, especificação arquitetural, implementação e validação experimental de um Sistema Acadêmico desenvolvido integralmente em Java puro (JDK 17+), sem o auxílio de ferramentas externas de construção ou *frameworks* de terceiros. O sistema modela o relacionamento muitos-para-muitos entre as entidades `Aluno` e `Disciplina` por meio da classe associativa `Matricula`, responsável por encapsular os atributos intrínsecos ao vínculo (identificador sequencial, data de realização e status) e orquestrar de forma atômica a ligação bidirecional e a ocupação de vagas. Adotou-se a metodologia de Desenvolvimento Orientado a Especificações (*Spec-Driven Development*), estruturada sobre quatro artefatos normativos (`AGENTS.md`, `ARCHITECTURE.md`, `TASKS.md` e `README.md`) e uma arquitetura estrita em quatro camadas funcionais (`model`, `repository`, `service` e `view`), além da camada de inicialização e verificação (`app`). Para assegurar a integridade referencial e de domínio em memória, foram formalizadas seis invariantes matemáticas (I1 a I6), protegidas em profundidade por construtores privados, método fábrica estático (`Matricula.criar`), visibilidade restrita ao pacote (*package-private*), coleções somente-leitura e tratamento de exceções verificadas (`MatriculaException`). A persistência de dados ocorre em arquivos CSV codificados em UTF-8 via API nativa `java.nio.file`, com geração automática de semente inicial (*seed*) e recálculo dinâmico de vagas ocupadas. A interface gráfica em Java Swing opera na *Event Dispatch Thread* e incorpora laços de validação reativa de entrada. A validação empírica por meio do executor automatizado `CasosDeTesteRunner` obteve 100% de aprovação nos 17 cenários de teste (CT01 a CT17), comprovando que a distribuição de responsabilidades entre classes associadas garante coesão, rastreabilidade histórica (*soft delete*) e evolução sustentável.

Palavras-chave: Programação Orientada a Objetos. Classe Associativa. Consistência Bidirecional. Arquitetura em Camadas. Java.

ABSTRACT

Proper modeling of object relationships is a cornerstone of Software Engineering and Object-Oriented Programming (OOP), preventing state anomalies and tight coupling typically found in procedural designs based on monolithic God Classes. This paper presents the design, architectural specification, implementation, and experimental validation of an Academic Management System built entirely in pure Java (JDK 17+), without external build tools or third-party frameworks. The system models the many-to-many relationship between `Aluno` (Student) and `Disciplina` (Course) through the `Matricula` (Enrollment) association class, which encapsulates link-specific attributes (sequential identifier, enrollment date, and lifecycle status) and atomically coordinates bidirectional linking and seat allocation. A Spec-Driven Development methodology was adopted, governed by four normative documents (`AGENTS.md`, `ARCHITECTURE.md`, `TASKS.md`, and `README.md`) and a strict four-layer unidirectional architecture (`model`, `repository`, `service`, and `view`), complemented by an entry-point package (`app`). Six formal domain invariants (I1 to I6) were established and enforced through defense-in-depth mechanisms: private constructors, a static factory method (`Matricula.criar`), package-private mutators, unmodifiable collections, and checked domain exceptions (`MatriculaException`). Persistence is implemented using UTF-8 CSV files via the native `java.nio.file` API, featuring deterministic seed generation and dynamic seat recounting upon reload. The Java Swing graphical user interface runs on the Event Dispatch Thread and features reactive input-validation loops. Empirical verification via the custom automated runner `CasosDeTesteRunner` achieved a 100% pass rate across all 17 test scenarios (CT01 to CT17), demonstrating that distributing responsibilities across associated classes yields high cohesion, historical preservation via soft delete, and maintainable software evolution.

Keywords: Object-Oriented Programming. Association Class. Bidirectional Consistency. Layered Architecture. Java.

SUMÁRIO

1 INTRODUÇÃO **4**
1.1 CONTEXTUALIZAÇÃO E DEFINIÇÃO DO PROBLEMA 4
1.2 OBJETIVOS 4
1.2.1 Objetivo geral **4**
1.2.2 Objetivos específicos **4**
1.3 JUSTIFICATIVA E ESCOPO RESTRITIVO 5
1.4 ORGANIZAÇÃO DO TRABALHO 5
2 FUNDAMENTAÇÃO TEÓRICA **6**
2.1 ENCAPSULAMENTO, IDENTIDADE E SEMÂNTICA DE OPERAÇÕES 6
2.2 ASSOCIAÇÕES MUITOS-PARA-MUITOS E CLASSES ASSOCIATIVAS NA UML 6
2.3 PADRÕES DE PROJETO E PROTEÇÃO DE INVARIANTES DE DOMÍNIO 7
2.4 ARQUITETURA EM CAMADAS E DESENVOLVIMENTO ORIENTADO A ESPECIFICAÇÕES 7
3 METODOLOGIA E ARQUITETURA DO SISTEMA **9**
3.1 METODOLOGIA DE DESENVOLVIMENTO EM ETAPAS E REGRAS DE OURO 9
3.2 ARQUITETURA EM CAMADAS UNIDIRECIONAIS E MATRIZ DE DEPENDÊNCIAS 9
3.3 MODELAGEM DETALHADA DO DOMÍNIO E CICLO DE VIDA DA MATRÍCULA 11
3.4 CONSISTÊNCIA EM MEMÓRIA E AS SEIS INVARIANTES FORMAIS (I1–I6) 12
3.5 REGRAS DE NEGÓCIO, ORDEM PRIORITÁRIA DE VALIDAÇÃO E PERSISTÊNCIA CSV 13
3.6 INTERFACE GRÁFICA SWING E VALIDAÇÃO REATIVA DE ENTRADAS 15
4 RESULTADOS EXPERIMENTAIS E DISCUSSÃO **17**
4.1 ARQUITETURA DO EXECUTOR DE TESTES AUTOMATIZADOS 17
4.2 ANÁLISE DOS 17 CASOS DE TESTE AUTOMATIZADOS (CT01 A CT17) 17
4.3 DISCUSSÃO: CLASSES ASSOCIADAS EM CAMADAS VERSUS CLASSE ÚNICA 18
5 CONCLUSÃO E TRABALHOS FUTUROS **20**
REFERÊNCIAS **21**

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO E DEFINIÇÃO DO PROBLEMA

O desenvolvimento de sistemas de informação voltados à gestão acadêmica envolve, em sua essência, a representação fidedigna de entidades interdependentes e de suas regras de negócio. Em cenários introdutórios de programação, observa-se com frequência a tendência de concentrar dados e comportamentos em uma única estrutura monolítica — conhecida na literatura de engenharia de software como *God Class* ou classe "faz-tudo" (Martin, 2017; Fowler, 2002). Nessa abordagem ingênua, relacionamentos entre alunos e disciplinas costumam ser mantidos por coleções paralelas desconectadas, tais como mapas de listas de códigos de disciplinas indexados pelo Registro Acadêmico (`Map<String, List<String>>`), tabelas de contagem de vagas (`Map<String, Integer>`) e dicionários de estados indexados por chaves compostas concatenadas.

Embora funcional em exemplos triviais, a ausência de associações explícitas entre objetos de domínio degrada rapidamente a coesão e compromete a integridade em memória. Quando o vínculo entre um estudante e uma turma e a respectiva contagem de vagas ocupadas residem em estruturas independentes, qualquer falha parcial durante uma operação de matrícula ou cancelamento pode produzir estados inconsistentes — por exemplo, decrementar a vaga disponível de uma disciplina sem registrar o vínculo correspondente no histórico do aluno, ou permitir matrículas duplicadas devido à falta de sincronização bidirecional (Larman, 2007). Além disso, atributos que pertencem propriamente à relação — como a data em que a inscrição foi efetuada e a situação cadastral do vínculo (ativa ou cancelada) — não possuem um local natural de armazenamento nem nas classes `Aluno` e `Disciplina` isoladamente.

Nesse contexto, a Programação Orientada a Objetos (POO) oferece mecanismos formais de modelagem estrutural e comportamental, destacando-se o conceito de **classe associativa** na Linguagem de Modelagem Unificada (UML) (Booch; Rumbaugh; Jacobson, 2006). Ao materializar o relacionamento muitos-para-muitos (`0..*` para `0..*`) entre `Aluno` e `Disciplina` por meio da classe associativa `Matricula`, torna-se possível encapsular os dados próprios do vínculo, preservar o histórico completo de movimentações por exclusão lógica (*soft delete*) e assegurar, em ponto único do código, que toda alteração de estado respeite invariantes rigorosas de consistência.

1.2 OBJETIVOS

1.2.1 Objetivo geral

Desenvolver e validar experimentalmente um sistema acadêmico em Java puro (JDK 17+), orientado a objetos e organizado em camadas, que represente as associações bidirecionais entre `Aluno`, `Disciplina` e a classe associativa `Matricula`, mantendo a consistência formal entre os objetos relacionados em memória e em persistência CSV, além de evidenciar os benefícios da divisão de responsabilidades entre classes associadas em comparação com a concentração de toda a lógica em uma única classe.

1.2.2 Objetivos específicos

Para alcançar o objetivo geral proposto, foram estabelecidos os seguintes objetivos específicos, alinhados à especificação técnica do projeto:

a. **Modelar** as classes de domínio `Aluno`, `Disciplina` e `Matricula` com encapsulamento estrito, utilizando `Matricula` como classe associativa e respeitando as multiplicidades `1` e `0..*` definidas no diagrama de classes UML;

b. **Implementar** as operações de realizar matrícula, cancelar matrícula, cadastrar, editar e remover alunos e disciplinas dinamicamente em tempo de execução, bem como listar as disciplinas ativas de cada aluno, garantindo a consistência bidirecional e a integridade referencial entre os objetos associados;

c. **Aplicar** regras de negócio que impeçam matrículas duplicadas, matrículas em disciplinas lotadas, cadastros com identificadores repetidos, redução de limite de vagas abaixo da ocupação vigente e exclusão de entidades com vínculos ativos, tratando as recusas por meio da exceção verificada personalizada `MatriculaException`;

d. **Organizar** a arquitetura do software em pacotes desacoplados e unidirecionais (`model`, `repository`, `service`, `view` e `app`), isolando a interface gráfica e a persistência da lógica de domínio;

e. **Persistir** o estado das entidades e de seus vínculos em arquivos de valores separados por ponto e vírgula (CSV) codificados em UTF-8 utilizando exclusivamente a API nativa `java.nio.file`, com geração automática de dados iniciais (*seed*);

f. **Construir** uma interface gráfica desktop responsiva em Java Swing (`TelaPrincipal`), executada na *Event Dispatch Thread* (EDT) e dotada de diálogos de entrada com laços de validação reativa que previnam o cancelamento abrupto por erros de digitação;

g. **Validar** o comportamento funcional e as invariantes estruturais do sistema por meio de uma suíte de 17 casos de teste automatizados em console (`CasosDeTesteRunner`), executada sobre diretório temporário isolado.

1.3 JUSTIFICATIVA E ESCOPO RESTRITIVO

A relevância pedagógica e científica deste trabalho reside na demonstração explícita dos fundamentos da orientação a objetos sem a intermediação de geradores de código ou abstrações opacas. No ecossistema corporativo moderno, o uso disseminado de bibliotecas como Lombok, Hibernate/JPA, Spring Framework e JUnit frequentemente oculta detalhes cruciais do ciclo de vida dos objetos, da proteção de coleções mutáveis e do controle de visibilidade entre pacotes.

Por essa razão, o projeto foi submetido a dez restrições arquiteturais inegociáveis ("Regras de Ouro", documentadas no artefato `AGENTS.md`): (R1) uso exclusivo de Java puro no padrão JDK 17+ (apenas pacotes `java.*` e `javax.swing.*`); (R2) proibição absoluta de sistemas de automação de construção como Maven ou Gradle; (R3) vedação a quaisquer bibliotecas ou *frameworks* externos; (R4) teto máximo de 10 arquivos `.java` (tendo o sistema completo sido construído com exatamente 9 arquivos); (R5) separação estrita de camadas sem ciclos de importação; (R6) fidelidade integral à especificação prévia (*Spec-Driven Development*); (R7) construção incremental verificável etapa por etapa; (R8) código didático documentado com Javadoc e marcadores semânticos em português; (R9) preservação determinística da semente de dados (*seed*); e (R10) compilação limpa sob verificação máxima do compilador (`javac -Xlint:all`) com zero erros e zero avisos.

1.4 ORGANIZAÇÃO DO TRABALHO

Além desta seção introdutória, o trabalho está estruturado em mais quatro seções. A Seção 2 apresenta a fundamentação teórica sobre encapsulamento, identidade de objetos, classes associativas na UML, padrões de projeto GoF/DDD e arquitetura em camadas. A Seção 3 detalha a metodologia *Spec-Driven Development*, a arquitetura em pacotes, o modelo de domínio, as seis invariantes formais de consistência, as regras de negócio, o mecanismo de persistência em CSV e a interface gráfica Swing. A Seção 4 expõe e discute os resultados obtidos na execução dos 17 casos de teste automatizados e analisa comparativamente o modelo proposto frente à abordagem de classe única. Por fim, a Seção 5 sintetiza as conclusões e aponta direções para trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 ENCAPSULAMENTO, IDENTIDADE E SEMÂNTICA DE OPERAÇÕES

A orientação a objetos estrutura programas a partir da interação entre cápsulas coesas de estado e comportamento (Deitel; Deitel, 2017). Segundo Bloch (2018), o encapsulamento — também denominado ocultação de informação — é o mecanismo central para desacoplar os componentes de um sistema, permitindo que cada classe proteja suas invariantes internas contra corrupção externa. Para que o encapsulamento seja efetivo na linguagem Java, não basta declarar os atributos como `private`; é igualmente indispensável impedir que referências para objetos mutáveis internos (como instâncias de `ArrayList`) escapem por meio de métodos de acesso (*getters*) desprotegidos, bem como recusar a criação indiscriminada de métodos modificadores genéricos (*setters*).

> Para que uma classe mantenha o controle sobre suas invariantes de estado, a acessibilidade de cada membro deve ser reduzida ao mínimo possível. Atributos nunca devem ser públicos, coleções internas jamais devem ser retornadas diretamente sem um invólucro não modificável ou cópia defensiva, e toda transição de estado deve ocorrer exclusivamente por meio de operações com significado explícito no domínio do problema, validando pré-condições antes de efetivar qualquer mutação (Bloch, 2018, p. 73, tradução nossa).

Na modelagem tática proposta pelo *Domain-Driven Design* (DDD), Evans (2003) distingue os objetos de domínio que possuem continuidade e identidade própria ao longo do tempo — denominados **Entidades** — daqueles definidos apenas pelos seus atributos. No domínio acadêmico investigado neste trabalho, `Aluno` e `Disciplina` configuram entidades clássicas: a identidade de um estudante é determinada univocamente pelo seu Registro Acadêmico (`ra`), enquanto a identidade de uma componente curricular é definida pelo seu código alfanumérico (`codigo`). Em termos de implementação em Java, isso exige que os identificadores `ra` e `codigo` sejam imutáveis (`private final`) e constituam o único critério utilizado na sobrescrita dos métodos `equals(Object)` e `hashCode()`.

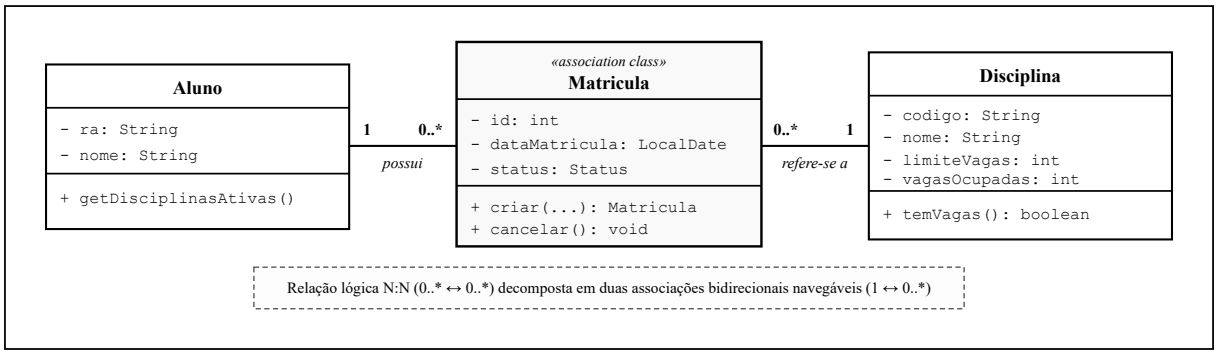
Em contrapartida, atributos descritivos ou operacionais — como o nome do aluno, o título da disciplina e o limite total de vagas ofertadas — podem sofrer retificações cadastrais durante a vida útil do registro. Todavia, em conformidade com os princípios de encapsulamento semântico, tais alterações não são expostas por métodos `setNome` ou `setLimiteVagas` arbitrários, mas sim por métodos de negócio explícitos (`alterarNome` e `alterarLimiteVagas`), os quais impõem validações imediatas de domínio (rejeição de cadeias vazias e proibição de reduzir a capacidade da turma para um valor inferior ao total de vagas já ocupadas).

2.2 ASSOCIAÇÕES MUITOS-PARA-MUITOS E CLASSES ASSOCIATIVAS NA UML

Na Linguagem de Modelagem Unificada (UML), uma **associação** descreve um relacionamento estrutural entre classes que especifica que objetos de uma espécie estão conectados a objetos de outra (Booch; Rumbaugh; Jacobson, 2006). A **multiplicidade** restringe formalmente quantos objetos de uma extremidade podem se vincular a uma única instância da extremidade oposta. No contexto universitário, um aluno pode matricular-se em zero ou várias disciplinas (`0..*`), ao passo que uma disciplina pode receber a inscrição de zero ou vários alunos (`0..*`), configurando logicamente um relacionamento muitos-para-muitos (*N:N*).

Quando um relacionamento *N:N* carrega informações próprias que não podem ser atribuídas exclusivamente a nenhum dos participantes isolados — tais como a data exata da inscrição (`dataMatricula`), o número de protocolo (`id`) e a situação de vigência (`status`) —, a especificação UML preconiza o emprego de uma **classe associativa** (Larman, 2007). A classe associativa `Matricula` decompõe a relação muitos-para-muitos original em duas associações bidirecionais de multiplicidade um-para-muitos (`1` para `0..*`): cada instância de `Aluno` associa-se a `0..*` instâncias de `Matricula`, e cada instância de `Disciplina` associa-se a `0..*` instâncias de `Matricula`, enquanto cada `Matricula` referencia exatamente um (`1`) `Aluno` e exatamente uma (`1`) `Disciplina`, conforme ilustrado na Figura 1.

Figura 1 — Diagrama UML simplificado do domínio evidenciando a classe associativa `Matricula` e as multiplicidades



Fonte: Elaborado pelos autores com base em `ARCHITECTURE.md` e `Apresentação.pdf` (2026).

2.3 PADRÕES DE PROJETO E PROTEÇÃO DE INVARIANTES DE DOMÍNIO

A implementação de associações bidirecionais em linguagens orientadas a objetos exige cuidado especial para evitar estados intermediários inválidos. Se o construtor de uma classe associativa for público ou se tentar publicar sua própria referência (`this`) para outras classes antes da conclusão da inicialização, incorre-se na vulnerabilidade conhecida como *this-escape*, alertada tanto por Bloch (2018) quanto pelo analisador estático do compilador Java (`-Xlint:this-escape`).

Para solucionar esse problema de forma elegante, o projeto aplica quatro padrões de projeto consagrados na literatura:

a. ***Static Factory Method*** **(Método Fábrica Estático)**: Em conformidade com o Item 1 de Bloch (2018), o construtor de `Matricula` é declarado como `private`, sendo a instanciação delegada exclusivamente ao método estático `Matricula.criar(id, aluno, disciplina, data)`. Esse método fábrica constrói o objeto completamente inicializado com status `ATIVA`, invoca primeiro `disciplina.adicionarMatricula(m)` (que ocupa a vaga ou aborta com exceção caso a turma esteja lotada) e somente após o sucesso dessa etapa executa `aluno.adicionarMatricula(m)`. Garante-se, assim, atomicidade transacional em memória: jamais existirá uma matrícula vinculada a apenas um dos lados;

b. ***Soft Delete*** **(Exclusão Lógica e Preservação Histórica)**: Em sistemas acadêmicos reais, o cancelamento de uma matrícula não deve destruir o registro físico do vínculo, sob pena de perda de auditoria histórica. Ao cancelar uma inscrição por meio de `matricula.cancelar()`, o objeto transiciona de `ATIVA` para `CANCELADA` e libera imediatamente a vaga na disciplina (`disciplina.liberarVaga()`), permanecendo armazenado nas coleções históricas;

c. ***Facade*** **(Fachada de Serviços)**: Segundo Gamma *et al.* (1995), o padrão *Facade* fornece uma interface unificada de nível superior para um conjunto de interfaces de um subsistema. A classe `SistemaAcademicoService` atua como fachada da camada de negócios, centralizando as validações de casos de uso e isolando a camada de apresentação (`TelaPrincipal`) dos detalhes de manipulação do modelo e do repositório;

d. ***Repository*** **(Repositório de Persistência)**: Conforme definido por Fowler (2002) e Evans (2003), o padrão *Repository* media entre o domínio e a camada de mapeamento de dados. A classe `ArquivoRepository` encapsula toda a tradução entre objetos em memória e linhas de texto nos arquivos CSV (`alunos.csv`, `disciplinas.csv` e `matriculas.csv`), mantendo o domínio livre de qualquer dependência de entrada e saída (I/O).

2.4 ARQUITETURA EM CAMADAS E DESENVOLVIMENTO ORIENTADO A ESPECIFICAÇÕES

A Arquitetura em Camadas (*Layered Architecture*) organiza o sistema em partições horizontais onde cada nível possui uma responsabilidade bem delimitada e depende unicamente dos níveis inferiores (Martin, 2017; Larman, 2007). No presente trabalho, o princípio de separação de conceitos (*Separation of Concerns*) impede que a interface gráfica (`view`) acesse diretamente os arquivos CSV (`repository`) ou altere o estado de entidades do domínio (`model`) sem passar pelo crivo de validação da camada de serviço (`service`). De modo simétrico, o pacote `model` desconhece completamente a existência de janelas Swing ou de fluxos de arquivos em disco.

Complementarmente à organização estrutural, adotou-se a abordagem de **Desenvolvimento Orientado a Especificações** (*Spec-Driven Development*). Nessa metodologia, a documentação técnica formal antecede a codificação e atua como contrato normativo verificável. Qualquer evolução funcional — como a introdução das operações de remoção com integridade referencial e de edição cadastral com validação reativa de diálogos — exige, obrigatoriamente, a atualização prévia dos diagramas UML, das tabelas de mensagens contratuais e do plano de testes nos documentos de especificação (`AGENTS.md`, `ARCHITECTURE.md`, `TASKS.md` e `README.md`) antes da modificação de qualquer linha de código Java.

3 METODOLOGIA E ARQUITETURA DO SISTEMA

3.1 METODOLOGIA DE DESENVOLVIMENTO EM ETAPAS E REGRAS DE OURO

O processo de engenharia de software adotado neste projeto foi guiado pelo princípio *Spec-Driven Development*, estabelecendo uma ordem estrita de precedência documental: `AGENTS.md` (regras inegociáveis de codificação e estilo) > `ARCHITECTURE.md` (especificação UML, assinaturas, invariantes e mensagens de erro) > `TASKS.md` (roteiro de construção incremental e verificação) > `README.md` (contextualização acadêmica e instruções operacionais).

A implementação foi decomposta em cinco etapas incrementais e independentemente verificáveis, conforme planejado em `TASKS.md`:

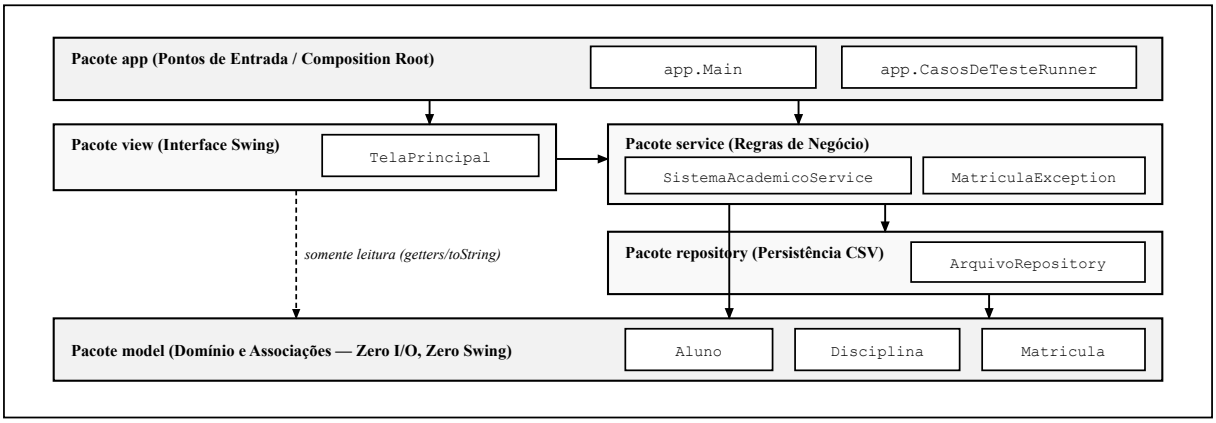
1. **Etapa 1 — Models e Associações**: Construção das classes `Aluno`, `Disciplina` e `Matricula` no pacote `model`, garantindo vínculo bidirecional atômico e controle de capacidade pelo próprio modelo, verificado via sessões interativas no `jshell`;
2. **Etapa 2 — Service e Regras de Negócio**: Implementação de `MatriculaException`, `SistemaAcademicoService` e da interface do `ArquivoRepository`, contemplando os casos de uso de matrícula, cancelamento, cadastro, edição, remoção e listagens imutáveis;
3. **Etapa 3 — Persistência CSV e Seed**: Finalização do `ArquivoRepository` utilizando `java.nio.file.Files` com codificação explícita `StandardCharsets.UTF_8`, separador ponto e vírgula (`;`) e geração automática dos dados de demonstração (*seed*);
4. **Etapa 4 — Script de Casos de Teste (`CasosDeTesteRunner`)**: Automação de 17 cenários de integração em console sobre diretório temporário, cobrindo fluxos de sucesso, bloqueios de regras de negócio, persistência e checagem sistemática das invariantes;
5. **Etapa 5 — Interface Gráfica Swing (`TelaPrincipal` e `Main`)**: Construção da janela desktop reativa com delegação estrita para a camada de serviço e validação segura de entradas do usuário.

Para tornar o código-fonte autodocumentado e rastreável durante inspeções acadêmicas e apresentações, convencionou-se a inserção de seis marcadores didáticos padronizados ao longo dos métodos: `// [ASSOCIAÇÃO]` (criação ou navegação de vínculos entre objetos), `// [CONSISTÊNCIA]` (manutenção explícita de uma invariante formal I1–I6), `// [ENCAPSULAMENTO]` (proteção de visibilidade, uso de `final` e coleções imutáveis), `// [REGRA DE NEGÓCIO]` (verificação de duplicidade, capacidade, existência ou integridade referencial), `// [CAMADAS]` (delegação entre pacotes distintos) e `// [PERSISTÊNCIA]` (operações de leitura/escrita em CSV e geração do *seed*).

3.2 ARQUITETURA EM CAMADAS UNIDIRECIONAIS E MATRIZ DE DEPENDÊNCIAS

O sistema encontra-se particionado em cinco pacotes com fluxo de dependência estritamente unidirecional (de cima para baixo), conforme representado na Figura 2. O pacote `app` contém exclusivamente os pontos de entrada (`Main` e `CasosDeTesteRunner`), atuando como raiz de composição (*Composition Root*) que instancia o repositório, injeta-o no serviço e inicializa a interface gráfica ou a bateria de testes.

Figura 2 — Arquitetura em camadas unidirecionais e fluxo de dependências entre os 5 pacotes do sistema



Fonte: Elaborado pelos autores com base em `ARCHITECTURE.md` (2026).

O Quadro 1 relaciona o inventário completo dos 9 arquivos `.java` que compõem o sistema, detalhando seu pacote, classificação arquitetural e responsabilidade única, bem como os conceitos de POO demonstrados em cada unidade de compilação. Em seguida, o Quadro 2 formaliza a matriz de dependências estritas verificada estaticamente.

Quadro 1 — Inventário completo dos 9 arquivos `.java` do sistema e suas responsabilidades

#	Arquivo (`src/`)	Pacote	Conceito de POO	Responsabilidade Principal
1	`model/Aluno.java`	`model`	Entidade, Encapsulamento, Associação `1..0..*`	Representa o estudante identificado pelo `ra`, mantém sua lista de matrículas e consulta disciplinas ativas.
2	`model/Disciplina.java`	`model`	Entidade, Defesa de Invariantes (I2, I3)	Representa a disciplina identificada por `codigo`, controla o limite e a ocupação de vagas e suas matrículas.
3	`model/Matricula.java`	`model`	Classe Associativa, *Static Factory Method*	Materializa o vínculo `Aluno`–`Disciplina`, armazena data e `Status` (`ATIVA`/`CANCELADA`) e liga ambos os lados.
4	`service/MatriculaException.java`	`service`	Exceção Verificada (*Checked Exception*)	Sinaliza violações de regras de negócio com mensagens padronizadas em português, exigindo tratamento explícito.
5	`service/SistemaAcademicoService.java`	`service`	Padrão *Facade*, Orquestração de Regras	Coordena os casos de uso (matrícula, cancelamento, cadastro, edição, remoção e consultas) e aciona a persistência.
6	`repository/ArquivoRepository.java`	`repository`	Padrão *Repository*, Isolamento de I/O	Converte objetos de domínio para linhas CSV (UTF-8), reconstrói os vínculos na carga e gera o *seed* inicial.
7	`view/TelaPrincipal.java`	`view`	Camada de Apresentação MVC/MVP (Swing)	Exibe o estado do sistema, captura eventos do usuário com validação reativa e delega todas as ações ao serviço.
8	`app/Main.java`	`app`	Injeção de Dependências Manual	Instancia `ArquivoRepository` e `SistemaAcademicoService`, carrega os dados e inicia a GUI na EDT.
9	`app/CasosDeTesteRunner.java`	`app`	Verificação Automatizada de Invariantes	Executa os 17 casos de teste em diretório temporário, audita as invariantes I1–I4 e gera relatório em console.

Fonte: Elaborado pelos autores com base em `AGENTS.md` e `ARCHITECTURE.md` (2026).

Quadro 2 — Matriz de dependências e controle de importações entre pacotes

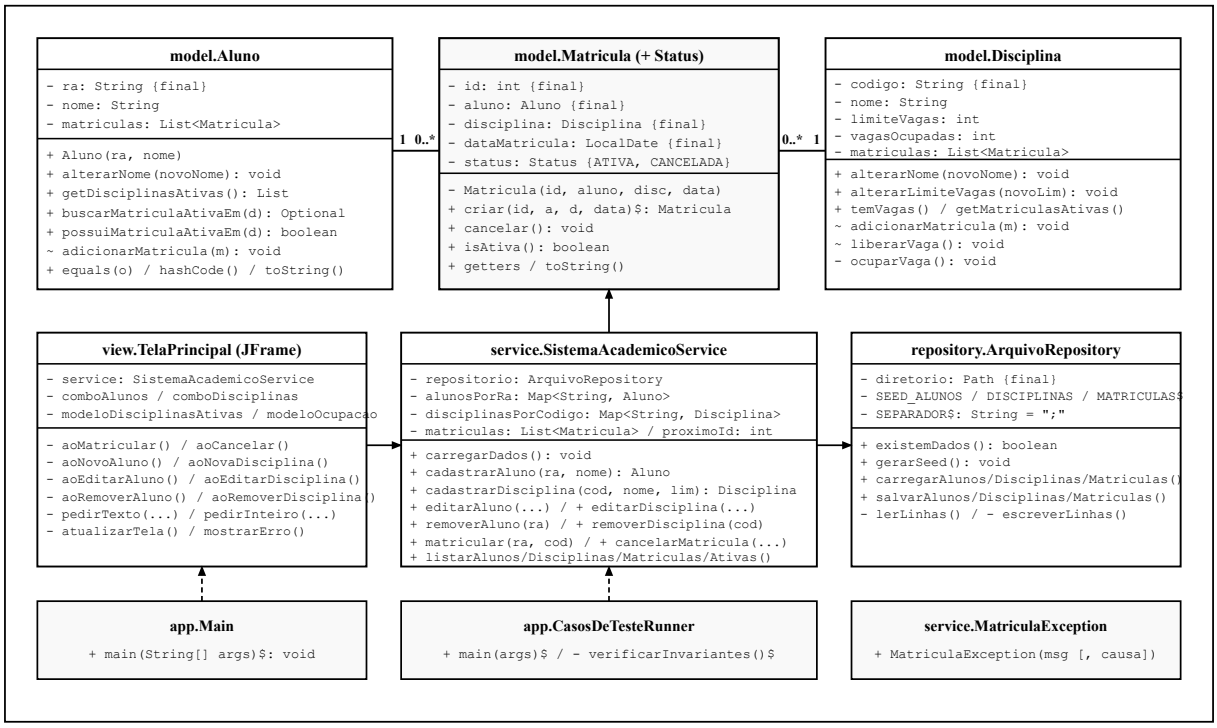
Pacote	Importações Permitidas	Importações Estritamente Proibidas
`model`	`java.util.*`, `java.time.*`	`service`, `repository`, `view`, `app`, `javax.swing.*`, `java.awt.*`, `java.io.*`, `java.nio.*`
`repository`	`model`, `java.nio.*`, `java.io.*`, `java.util.*`, `java.time.*`	`service`, `view`, `app`, `javax.swing.*`, `java.awt.*`
`service`	`model`, `repository`, `java.util.*`, `java.time.*`, `java.io.IOException`	`view`, `app`, `javax.swing.*`, `java.awt.*`
`view`	`service`, `model` (somente leitura), `javax.swing.*`, `java.awt.*`, `java.util.*`	`repository`, `java.io.*`, `java.nio.*`
`app`	`model`, `repository`, `service`, `view` e biblioteca padrão	Nenhuma restrição interna (atua apenas como ponto de entrada).

Fonte: Elaborado pelos autores com base na Seção 3 de `AGENTS.md` (2026).

3.3 MODELAGEM DETALHADA DO DOMÍNIO E CICLO DE VIDA DA MATRÍCULA

O diagrama de classes UML completo do sistema é apresentado na Figura 3, contemplando os atributos, visibilidades (+ público, – privado, ~ pacote e $ estático) e assinaturas de métodos das 9 classes.

Figura 3 — Diagrama de classes UML completo do Sistema Acadêmico



Fonte: Elaborado pelos autores com base na Seção 3 de `ARCHITECTURE.md` (2026).

O núcleo de consistência do modelo reside na coordenação entre `Matricula.criar(...)`, `Disciplina.adicionarMatricula(...)` e `Matricula.cancelar()`, cujo código-fonte real é reproduzido no Quadro 3. Observe-se que, durante a criação da matrícula, `disciplina.adicionarMatricula(matricula)` é invocado **antes** de `aluno.adicionarMatricula(matricula)`. Como `Disciplina.adicionarMatricula` executa primeiramente `ocuparVaga()` (que lança `IllegalStateException` caso `vagasOcupadas >= limiteVagas`), uma eventual falha interrompe o fluxo antes que a lista da disciplina ou a lista do aluno sejam alteradas, garantindo atomicidade absoluta.

Quadro 3 — Trecho de código-fonte de `model/Matricula.java` demonstrando a fábrica atômica e o cancelamento

```
public static Matricula criar(int id, Aluno aluno, Disciplina disciplina, LocalDate data) {
    Matricula matricula = new Matricula(id, aluno, disciplina, data);

    // [CONSISTÊNCIA] I1/I2: Vincula primeiro a disciplina (que valida e ocupa a vaga).
    // Se a disciplina estiver lotada, lança exceção antes de vincular ao aluno, preservando a
atomicidade.
    disciplina.adicionarMatricula(matricula);

    // [ASSOCIAÇÃO] Registra a matrícula no aluno após confirmação da vaga
    aluno.adicionarMatricula(matricula);

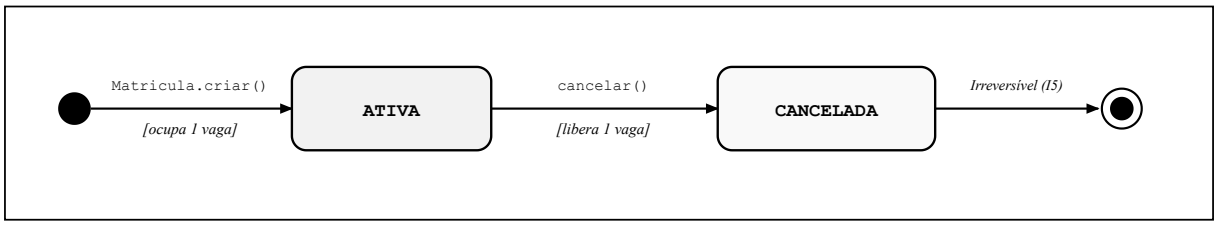
    return matricula;
}

public void cancelar() {
    if (status == Status.CANCELADA) {
        throw new IllegalStateException("A matrícula #" + id + " já está cancelada.");
    }
    // [CONSISTÊNCIA] I2/I5: Altera o status e libera a vaga na disciplina associada de forma
coordenada
    this.status = Status.CANCELADA;
    disciplina.liberarVaga();
}
```

Fonte: Extraído de `src/model/Matricula.java` (linhas 77–102) (2026).

O ciclo de vida de uma instância de `Matricula` segue uma máquina de estados finitos estritamente acíclica, ilustrada na Figura 4. Uma matrícula nasce no estado `ATIVA` (ocupando 1 vaga) e pode transicionar uma única vez para o estado terminal `CANCELADA` (liberando 1 vaga). Não existe método de reativação: caso o mesmo estudante deseje cursar novamente a disciplina após um cancelamento, um novo objeto `Matricula` com novo `id` sequencial é instanciado, preservando-se o registro cancelado anterior para fins de histórico (validado pelo caso de teste CT08).

Figura 4 — Diagrama de estados UML do ciclo de vida de um objeto `Matricula`



Fonte: Elaborado pelos autores com base na Seção 4.3 de `ARCHITECTURE.md` (2026).

3.4 CONSISTÊNCIA EM MEMÓRIA E AS SEIS INVARIANTES FORMAIS (I1–I6)

Uma **invariante de domínio** é uma asserção lógica que deve permanecer verdadeira antes e depois da execução de qualquer operação pública do sistema. Formalmente, seja A o conjunto de alunos cadastrados, D o conjunto de disciplinas ofertadas e M o conjunto global de matrículas. Para toda disciplina $d \in D$, a quantidade de vagas ocupadas `vagasOcupadas`(d) e o limite máximo `limiteVagas`(d) satisfazem a relação expressa na Equação (1):

$$0 \leq vagasOcupadas(d) = |\{\, m \in M(d) \mid status(m) = ATIVA \,\}| \leq limiteVagas(d) \tag{1}$$

O Quadro 4 apresenta as seis invariantes formais (I1 a I6) especificadas na arquitetura e nos slides da apresentação do projeto, acompanhadas das classes e mecanismos responsáveis por sua preservação.

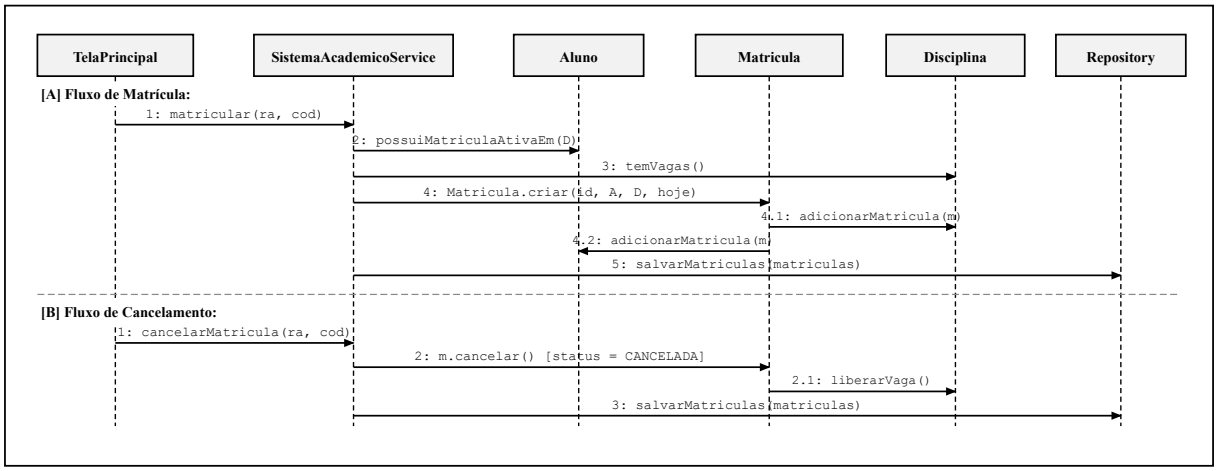
Quadro 4 — As seis invariantes formais de consistência do Sistema Acadêmico (I1 a I6)

ID	Denominação	Definição Formal da Invariante	Mecanismo Garantidor no Código
I1	Consistência Bidirecional	Toda `Matricula m` pertence simultaneamente a `m.getAluno().getMatriculas()` e a `m.getDisciplina().getMatriculas()`.	Ponto único de vínculo em `Matricula.criar(...)` + coleções não modificáveis.
I2	Contagem Exata de Vagas	`disciplina.getVagasOcupadas()` é exatamente igual ao número de matrículas com status `ATIVA` na disciplina.	`Disciplina.adicionarMatricula` (incrementa) e `Matricula.cancelar` (decrementa).
I3	Limite de Capacidade	Para toda disciplina, `0 <= vagasOcupadas <= limiteVagas` em qualquer instante de execução.	Validação no Service + defesa interna em `ocuparVaga`, `liberarVaga` e `alterarLimiteVagas`.
I4	Unicidade de Matrícula Ativa	Existe no máximo uma única matrícula com status `ATIVA` para cada par `(Aluno, Disciplina)`.	Verificação prévia via `aluno.possuiMatriculaAtivaEm(disciplina)` no Service.
I5	Irreversibilidade do Cancelamento	Uma matrícula com status `CANCELADA` jamais retorna ao estado `ATIVA`; rematrícula gera novo objeto.	Guarda de estado em `Matricula.cancelar()` e ausência de *setter* ou método de reativação.
I6	Identificador Único Sequencial	Os atributos `id` das matrículas são estritamente positivos, únicos e crescentes mesmo após recarga de disco.	Cálculo de `proximoIdMatricula = maiorId + 1` em `carregarDados()` no Service.

Fonte: Elaborado pelos autores com base na Seção 5 de `ARCHITECTURE.md` e página 6 de `Apresentação.pdf` (2026).

A proteção dessas seis invariantes apoia-se em cinco mecanismos de defesa em profundidade: (1) **ponto único de vínculo** via método fábrica `Matricula.criar(...)`; (2) **visibilidade restrita de pacote (*package-private*)** nos métodos `adicionarMatricula` e `liberarVaga`, impedindo que as camadas `service` ou `view` manipulem ponteiros ou contadores diretamente; (3) **coleções não modificáveis** retornadas por todos os métodos de consulta (`Collections.unmodifiableList` no modelo e `List.copyOf` no serviço), lançando `UnsupportedOperationException` diante de qualquer tentativa externa de `add`, `remove` ou `clear`; (4) **atributos `final`** nas referências estruturais; e (5) **dupla barreira de validação** (exceção verificada amigável `MatriculaException` no serviço e exceções de tempo de execução `IllegalArgumentException` / `IllegalStateException` no modelo). A Figura 5 ilustra a sequência completa de mensagens trocadas entre as camadas durante a realização e o cancelamento de uma matrícula.

Figura 5 — Diagrama de sequência UML das operações de realizar matrícula e cancelar matrícula



Fonte: Elaborado pelos autores com base nas Seções 5.2 e 5.3 de `ARCHITECTURE.md` (2026).

3.5 REGRAS DE NEGÓCIO, ORDEM PRIORITÁRIA DE VALIDAÇÃO E PERSISTÊNCIA CSV

Na camada `service`, o método `SistemaAcademicoService.matricular(ra, codigoDisciplina)` aplica uma sequência determinística de quatro etapas de validação antes de instanciar a classe associativa: (Etapa 01) verificação de existência do aluno pelo RA; (Etapa 02) verificação de existência da disciplina pelo código; (Etapa 03) verificação de

duplicidade de matrícula ativa (`aluno.possuiMatriculaAtivaEm(disciplina)`); e (Etapa 04) verificação de **disponibilidade de vagas** (`disciplina.temVagas()`).

A precedência da Etapa 03 sobre a Etapa 04 possui justificativa semântica rigorosa: se um aluno que já está regularmente matriculado em uma turma lotada tentar matricular-se novamente nessa mesma disciplina, a causa primária de rejeição do domínio é o fato de ele já possuir vínculo ativo, e não a lotação da turma. De forma análoga, no método `editarDisciplina(codigo, novoNome, novoLimiteVagas)`, todas as quatro validações (campos não vazios, `novoLimiteVagas > 0`, existência da disciplina e `novoLimiteVagas >= disciplina.getVagasOcupadas()`) são executadas **antes** de qualquer chamada a `alterarNome` ou `alterarLimiteVagas`, evitando que uma recusa deixe a entidade parcialmente editada. O Quadro 5 sintetiza o contrato de mensagens padronizadas lançadas via `MatriculaException`.

Quadro 5 — Contrato de regras de negócio e mensagens padronizadas de `MatriculaException`

Situação de Negócio	Método Verificador	Mensagem Contratual (`MatriculaException`)
RA inexistente, nulo ou vazio na busca	`buscarAluno`	`Aluno com RA '<ra>' não encontrado.`
Código inexistente, nulo ou vazio	`buscarDisciplina`	`Disciplina com código '<codigo>' não encontrada.`
Matrícula ativa duplicada (I4)	`matricular`	`O aluno <nome> já possui matrícula ativa em <codigo>.`
Disciplina sem vagas disponíveis (I3)	`matricular`	`A disciplina <codigo> está lotada (<ocupadas>/<limite> vagas ocupadas).`
Cancelamento sem matrícula ativa	`cancelarMatricula`	`O aluno <nome> não possui matrícula ativa em <codigo>.`
Duplicidade de RA ou código no cadastro	`cadastrarAluno` / `Disciplina`	`Aluno com RA '<ra>' já está cadastrado.` / `Disciplina com código '<codigo>' já está cadastrada.`
Campos obrigatórios nulos ou em branco	`cadastrar*` / `editar*`	`O [RA/nome do aluno | código/nome da disciplina] não pode ser nulo ou vazio.`
Limite de vagas menor ou igual a zero	`cadastrar*` / `editarDisciplina`	`O limite de vagas deve ser maior que zero.`
Redução de limite abaixo da ocupação (I3)	`editarDisciplina`	`O novo limite de vagas (<novo>) não pode ser menor que o total de vagas já ocupadas (<ocupadas>).`
Remoção de entidade com vínculo ativo	`removerAluno` / `Disciplina`	`O aluno <nome> [A disciplina <codigo>] não pode ser removido(a) pois possui matrículas ativas.`

Fonte: Elaborado pelos autores com base na Seção 6 de `ARCHITECTURE.md` (2026).

No subsistema de persistência (`ArquivoRepository`), os dados são gravados na pasta `dados/` em três arquivos CSV separados por ponto e vírgula (`;`) e codificados em UTF-8: `alunos.csv` (cabeçalho `ra;nome`), `disciplinas.csv` (cabeçalho `codigo;nome;limiteVagas`) e `matriculas.csv` (cabeçalho `id;ra;codigoDisciplina;data;status`). Uma decisão arquitetural fundamental (Decisão D3 de `ARCHITECTURE.md`) consiste em **não persistir** o atributo `vagasOcupadas` no arquivo `disciplinas.csv`. Por tratar-se de um atributo derivado das matrículas ativas, sua gravação em disco introduziria redundância e risco de anomalia de atualização. Em vez disso, durante a rotina `carregarDados()`, as disciplinas são instanciadas com `vagasOcupadas = 0` e, à medida que cada linha de `matriculas.csv` é processada por `Matricula.criar(...)` (e seguida imediatamente por `m.cancelar()` caso o status lido seja `CANCELADA`), a contagem exata de vagas é reconstruída organicamente pelo próprio modelo.

Quando o diretório de armazenamento não contém os três arquivos, o repositório executa automaticamente `gerarSeed()`, populando o estado inicial determinístico composto por 5 alunos, 4 disciplinas e 8 matrículas (7 ativas e 1 cancelada), cujos valores quantitativos e estruturais são apresentados nas Tabelas 1, 2 e 3 (formatadas segundo o padrão aberto nas laterais do IBGE).

Tabela 1 — Alunos integrantes da carga inicial determinística (*seed*) em `alunos.csv`

RA	Nome do Aluno	Matrículas Totais	Matrículas Ativas
2024001	Ana Souza	2	2
2024002	Bruno Lima	2	2
2024003	Carla Mendes	2	1
2024004	Diego Rocha	2	2
2024005	Elisa Ferreira	0	0
Total de alunos no *seed*: 5		**8**	**7**

Fonte: Dados extraídos de `src/repository/ArquivoRepository.java` e `README.md` (2026).

Tabela 2 — Disciplinas ofertadas na carga inicial (*seed*) em `disciplinas.csv` e ocupação derivada

Código	Nome da Disciplina	Vagas Ocupadas	Limite de Vagas	Taxa de Ocupação
POO101	Programação Orientada a Objetos	3	30	10,0%
BD201	Banco de Dados	1	25	4,0%
ED301	Estruturas de Dados	1	20	5,0%
LAB401	Laboratório de Robótica	2	2	100,0% (LOTADA)
Total de disciplinas no *seed*: 4		**7**	**77**	**9,1%**

Fonte: Dados extraídos de `src/repository/ArquivoRepository.java` e `README.md` (2026).

Tabela 3 — Registro histórico de matrículas iniciais (*seed*) em `matriculas.csv`

ID	RA do Aluno	Nome do Aluno	Disciplina	Data ISO	Status
1	2024001	Ana Souza	POO101	2026-02-10	ATIVA
2	2024001	Ana Souza	BD201	2026-02-10	ATIVA
3	2024002	Bruno Lima	POO101	2026-02-11	ATIVA
4	2024002	Bruno Lima	LAB401	2026-02-11	ATIVA
5	2024003	Carla Mendes	LAB401	2026-02-12	ATIVA
6	2024003	Carla Mendes	ED301	2026-02-12	CANCELADA
7	2024004	Diego Rocha	ED301	2026-02-13	ATIVA
8	2024004	Diego Rocha	POO101	2026-02-13	ATIVA

Fonte: Dados extraídos de `src/repository/ArquivoRepository.java` (linhas 48–58) (2026).

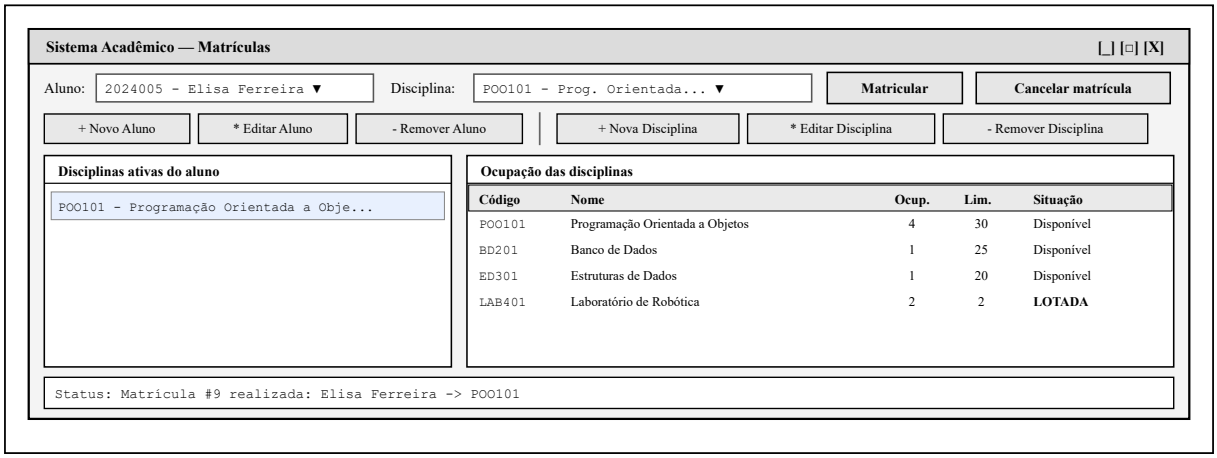
3.6 INTERFACE GRÁFICA SWING E VALIDAÇÃO REATIVA DE ENTRADAS

A camada de apresentação (`view.TelaPrincipal`) estende `javax.swing.JFrame` e é instanciada exclusivamente dentro da *Event Dispatch Thread* (EDT) por meio de `SwingUtilities.invokeLater` em `app.Main`. O *layout* utiliza gerenciadores nativos (`BorderLayout`, `GridLayout` e `FlowLayout`): no topo (`NORTH`), duas faixas horizontais acomodam os seletores `JComboBox<Aluno>` e `JComboBox<Disciplina>` e oito botões de comando (`Matricular`, `Cancelar matrícula`, `+ Novo Aluno`, `* Editar Aluno`, `- Remover Aluno`, `+ Nova Disciplina`, `* Editar Disciplina` e `- Remover Disciplina`); no centro (`CENTER`), dividem espaço a lista de disciplinas ativas do aluno selecionado (`JList<Disciplina>`) e a tabela não editável de ocupação geral das disciplinas (`JTable`); no rodapé (`SOUTH`), uma barra de status (`JLabel`) informa o resultado da última operação realizada, conforme representado na Figura 6.

Um aprimoramento importante de usabilidade e robustez implementado em `TelaPrincipal` consiste nos métodos privados auxiliares `pedirTexto(titulo, mensagem, valorInicial)` e `pedirInteiro(titulo, mensagem, valorInicial)`. Em implementações ingênuas com `JOptionPane.showInputDialog`, caso o usuário digite acidentalmente um caractere alfabético ou um número não positivo ao preencher o limite de vagas de uma disciplina, a captura de `NumberFormatException` encerra e aborta todo o fluxo de cadastro ou edição. Com o uso de um laço `while (true)` nos métodos auxiliares, entradas em branco ou valores não numéricos/não positivos exibem um diálogo de alerta

(`JOptionPane.WARNING_MESSAGE`) explicando o problema e reabrem imediatamente o campo mantendo o texto digitado para correção, abortando a operação somente quando o usuário aciona explicitamente o botão *Cancelar* ou fecha a caixa de diálogo (retorno `null`).

Figura 6 — Representação visual da interface gráfica Swing (`TelaPrincipal`) após matrícula de Elisa Ferreira em `POO101`



Fonte: Elaborado pelos autores com base em `src/view/TelaPrincipal.java` e Seção 4.7 de `ARCHITECTURE.md` (2026).

4 RESULTADOS EXPERIMENTAIS E DISCUSSÃO

4.1 ARQUITETURA DO EXECUTOR DE TESTES AUTOMATIZADOS

Em virtude da proibição do uso de bibliotecas externas de teste como JUnit ou TestNG (Regra R3 de `AGENTS.md`), foi projetado um motor próprio de verificação automatizada na classe `app.CasosDeTesteRunner`. O executor define uma interface funcional privada aninhada (`@FunctionalInterface private interface Teste { void executar() throws Exception; }`) e estrutura a execução sob quatro princípios de engenharia de testes:

a. **Isolamento de Ambiente em Diretório Temporário**: No início da execução, `Files.createTempDirectory("academico-teste")` aloca uma pasta efêmera no sistema operacional. Assim, a geração de *seed*, as gravações e as recargas de arquivos CSV ocorrem em isolamento completo, sem jamais interferir na pasta `dados/` utilizada pela aplicação gráfica;

b. **Asserções Independentes de Flags da JVM**: Em vez de utilizar a palavra-chave nativa `assert` do Java (que permanece desabilitada por padrão na máquina virtual caso a flag `-ea` não seja informada), o método auxiliar `verificar(condicao, mensagemFalha)` lança incondicionalmente `AssertionError` sempre que uma condição esperada não se confirma;

c. **Captura Tipada de Exceções de Negócio**: O método auxiliar `esperarExcecao(Teste acao)` executa o bloco lambda fornecido e exige que uma instância de `MatriculaException` seja lançada, devolvendo-a para inspeção exata da mensagem textual contratual;

d. **Auditoria Contínua das Invariantes Formais**: Ao final de cada um dos 17 casos de teste, o método `verificarInvariantes(service)` percorre todas as matrículas e disciplinas em memória para auditar programaticamente as invariantes I1 (presença simultânea nas coleções de `Aluno` e `Disciplina`), I2 (igualdade estrita entre `vagasOcupadas` e a contagem de matrículas ativas), I3 (respeito aos limites `0 <= vagasOcupadas <= limiteVagas`) e I4 (ausência de pares duplicados `ra:codigo` ativos usando um `HashSet<String>`).

4.2 ANÁLISE DOS 17 CASOS DE TESTE AUTOMATIZADOS (CT01 A CT17)

A compilação do projeto com verificação estrita do compilador (`javac -Xlint:all -encoding UTF-8`) transcorreu com zero erros e zero avisos. Em seguida, a execução da suíte `app.CasosDeTesteRunner` avaliou sequencialmente os 17 casos de teste planejados em `TASKS.md`, compartilhando o estado evolutivo para formar uma narrativa completa de uso do sistema. O Quadro 6 apresenta os resultados detalhados obtidos na execução real.

Quadro 6 — Resultados experimentais obtidos na execução dos 17 casos de teste automatizados (`CasosDeTesteRunner`)

Caso	Cenário Avaliado	Resultado Esperado	Resultado Obtido na Execução	Status
CT01	Carga inicial (*seed*) e consistência	5 alunos, 4 disciplinas, 8 matrículas (7 ativas); `LAB401` 2/2; invariantes OK.	5 alunos, 4 disciplinas, 8 matrículas (7 ativas); invariantes I1–I4 OK.	PASSOU
CT02	Matrícula válida	Matrícula `#9 ATIVA` (Elisa → `POO101`); ocupação de 3/30 para 4/30.	`#9 Elisa Ferreira -> POO101 [ATIVA]`; `POO101` 4/30; vínculo bidirecional OK.	PASSOU
CT03	Bloqueio de matrícula duplicada	`MatriculaException` ao matricular Ana Souza novamente em `POO101`.	`"O aluno Ana Souza já possui matrícula ativa em POO101."`; permanece 4/30.	PASSOU
CT04	Bloqueio por disciplina lotada	`MatriculaException` ao matricular Elisa em `LAB401` (2/2 vagas).	`"A disciplina LAB401 está lotada (2/2 vagas ocupadas)."`; permanece 2/2.	PASSOU
CT05	Cancelamento de matrícula	Matrícula `#4` (Bruno em `LAB401`) passa a `CANCELADA`; vagas 2/2 → 1/2.	Matrícula `#4 CANCELADA`; `LAB401` 1/2; histórico de Bruno preservado (2 reg.).	PASSOU
CT06	Reaproveitamento da vaga liberada	Elisa matricula-se em `LAB401` gerando `#10 ATIVA`; vagas 1/2 → 2/2.	`#10 Elisa Ferreira -> LAB401 [ATIVA]`; `LAB401` volta a 2/2.	PASSOU
CT07	Cancelamento de matrícula inexistente	`MatriculaException` ao tentar cancelar Elisa em `ED301`.	`"O aluno Elisa Ferreira não possui matrícula ativa em ED301."`	PASSOU

Caso	Cenário Avaliado	Resultado Esperado	Resultado Obtido na Execução	Status
CT08	Rematrícula após cancelamento	Carla em `ED301` (onde `#6` é `CANCELADA`) gera nova `#11 ATIVA`; `#6` intacta.	`#11 Carla Mendes -> ED301 [ATIVA]`; `ED301` 2/20; 3 matrículas no aluno.	PASSOU
CT09	Listagem de disciplinas ativas	`listarDisciplinasAtivas("2024001")` retorna `[POO101, BD201]` na ordem.	`[POO101, BD201]` na exata ordem cronológica de matrícula.	PASSOU
CT10	Aluno e disciplina inexistentes	Exceções padronizadas para RA `9999999` e código `XYZ999`.	Mensagens padronizadas validadas; total permanece em 11 matrículas.	PASSOU
CT11	Persistência e recarga de CSV	Novo `service` no mesmo diretório recarrega 11 matrículas (9 ativas) e gera `#12`.	11 matrículas recarregadas do disco; nova matrícula `#12` criada; `BD201` 2/25.	PASSOU
CT12	Encapsulamento protege a consistência	`UnsupportedOperationException` ao tentar `clear()` nas 3 listas expostas.	`UnsupportedOperationException` nas 3 tentativas; coleções internas intactas.	PASSOU
CT13	Cadastro dinâmico, matrícula e persistência	Cadastrar Fernanda (`2024006`) e `WEB501` (2 vagas), matricular (`#13`) e recarregar.	Aluno e disciplina cadastrados; matrícula `#13` criada e recarregada dos 3 CSVs.	PASSOU
CT14	Bloqueio de remoção com vínculos ativos	`MatriculaException` ao tentar remover Fernanda e `WEB501` com matrícula ativa.	Bloqueio de remoção validado com mensagens contratuais; entidades preservadas.	PASSOU
CT15	Remoção válida, expurgo e persistência	Cancelar matrícula `#13`, remover Fernanda e `WEB501`, expurgar histórico e recarregar.	Entidades removidas; CSVs atualizados e recarregados sem referências órfãs (12 mat.).	PASSOU
CT16	Edição de aluno e persistência	Alterar nome de Diego (`2024004`) para `"Diego R. Rocha"`; bloquear nome vazio; recarregar.	Nome atualizado e persistido no CSV; bloqueio de vazio OK; vínculos `#7` e `#8` intactos.	PASSOU
CT17	Edição de disciplina e defesa de limite (I3)	Expandir `ED301` para 25 vagas; bloquear redução de `LAB401` (2/2) para 1 vaga.	`ED301` atualizado (2/25) e persistido; redução abaixo da ocupação em `LAB401` bloqueada.	PASSOU

Fonte: Saída experimental gerada por `java -cp bin app.CasosDeTesteRunner` (2026).

A Tabela 4 consolida quantitativamente os resultados da verificação experimental agrupados por categoria funcional de requisito. Todos os 17 casos de teste obtiveram aprovação integral (código de saída `0`), sem nenhuma violação de invariante detectada nas 17 auditorias automáticas executadas.

Tabela 4 — Síntese quantitativa de cobertura e resultados por categoria funcional de teste

Categoria Funcional de Verificação	Casos Associados	Executados	Aprovados	Taxa (%)
Carga de *seed*, persistência CSV e recarga	CT01, CT11	2	2	100,0%
Ciclo de vida de matrícula, cancelamento e rematrícula	CT02, CT05, CT06, CT08, CT09	5	5	100,0%
Bloqueios de regras de negócio (duplicidade, lotação, inexistência)	CT03, CT04, CT07, CT10	4	4	100,0%
Blindagem por encapsulamento (coleções somente-leitura)	CT12	1	1	100,0%
Cadastro dinâmico, edição cadastral e remoção íntegra	CT13, CT14, CT15, CT16, CT17	5	5	100,0%
Total Geral da Bateria de Testes Automatizados		**17**	**17**	**100,0%**

Fonte: Elaborado pelos autores com base nos resultados de `CasosDeTesteRunner` (2026).

4.3 DISCUSSÃO: CLASSES ASSOCIADAS EM CAMADAS VERSUS CLASSE ÚNICA

Os resultados empíricos obtidos nos casos CT01 a CT17 permitem contrastar objetivamente a modelagem orientada a objetos por classes associadas e camadas frente à abordagem monolítica de uma única classe "faz-tudo" (*God Class*). Em uma classe monolítica hipotética que armazenasse relacionamentos em mapas paralelos desconectados (como `Map<String,`

`List<String>> disciplinasPorAluno` e `Map<String, Integer> vagasPorDisciplina`), a adição de operações como edição de capacidade (CT17) ou exclusão de entidades com histórico cancelado (CT15) exigiria varreduras manuais propensas a falhas de sincronismo em múltiplos pontos do código.

Em contrapartida, no modelo desenvolvido neste trabalho, cada classe possui um único eixo de mudança (Princípio da Responsabilidade Única — SRP, de Martin, 2017): `Matricula` governa a atomicidade do vínculo e a transição de status; `Disciplina` protege o invariante de capacidade (I3); `Aluno` organiza suas próprias inscrições; `SistemaAcademicoService` orquestra as políticas de aplicação; `ArquivoRepository` isola a serialização em CSV; e `TelaPrincipal` cuida exclusivamente da interação visual. O Quadro 7 sintetiza a comparação sistemática entre as duas arquiteturas segundo oito critérios de engenharia de software.

Quadro 7 — Comparação multicritério entre a abordagem de classe única e o modelo de classes associadas em camadas

Critério	Abordagem de Classe Única ("Faz-Tudo")	Classes Associadas em Camadas (Este Projeto)
Coesão	Baixa: mistura estruturas de dados, regras de validação, I/O de arquivos e componentes gráficos Swing.	Alta: cada classe e pacote possui responsabilidade única e claramente delimitada.
Consistência	Frágil: mapas paralelos podem divergir caso uma exceção ocorra no meio de uma atualização.	Forte: vínculo bidirecional e contagem de vagas mudam atomicamente em `Matricula.criar` e `cancelar`.
Dados do Vínculo	Espalhados em mapas auxiliares indexados por chaves concatenadas (ex.: `"2024001:POO101"`).	Encapsulados naturalmente como atributos próprios da classe associativa `Matricula`.
Histórico	Cancelamento costuma remover a chave do mapa, destruindo o registro histórico da inscrição.	Preservado via *soft delete* (`Status.CANCELADA`), permitindo rematrícula limpa (CT08).
Testabilidade	Baixa: difícil testar regras de negócio sem acionar janelas gráficas ou sobrescrever os arquivos reais.	Elevada: camada `service` testada em console sobre diretório temporário em segundos.
Evolução	Custosa: incluir "nota final" ou "frequência" exige criar novos mapas paralelos e alterar toda a classe.	Linear: basta adicionar novos atributos e métodos de negócio diretamente na classe `Matricula`.
Encapsulamento	Inexistente internamente: qualquer método da classe única pode corromper qualquer estrutura.	Rigoroso: visibilidade de pacote (~) e listas imutáveis bloqueiam mutações externas (CT12).
Trabalho em Equipe	Alto índice de conflitos de versão (todos os integrantes editam o mesmo arquivo monolítico).	Divisão modular: cada integrante pode desenvolver e revisar uma camada com contrato definido.

Fonte: Elaborado pelos autores com base na Seção 8 de `ARCHITECTURE.md` (2026).

5 CONCLUSÃO E TRABALHOS FUTUROS

Este trabalho apresentou o projeto arquitetural, a implementação em Java puro (JDK 17+) e a validação experimental de um Sistema Acadêmico voltado à demonstração prática e rigorosa de associações entre classes na Programação Orientada a Objetos. Ao decompor o relacionamento muitos-para-muitos entre `Aluno` e `Disciplina` por meio da classe associativa `Matricula`, comprovou-se que atributos próprios do vínculo — como o identificador sequencial, a data de inscrição e o status de vigência — encontram representação coesa e segura sem recorrer a estruturas paralelas frágeis.

Todos os sete objetivos específicos estabelecidos foram integralmente cumpridos. A adoção da metodologia *Spec-Driven Development* garantiu sincronismo de 100% entre a documentação normativa (`AGENTS.md`, `ARCHITECTURE.md`, `TASKS.md` e `README.md`) e os 9 arquivos `.java` finais, os quais compilam sem qualquer aviso sob a verificação estrita `javac -Xlint:all`. As seis invariantes formais de consistência (I1 a I6) mostraram-se inabaláveis frente a todas as operações de matrícula, cancelamento, rematrícula, cadastro dinâmico, edição cadastral com proteção de capacidade ocupada, remoção com verificação de integridade referencial e reconstrução a partir de arquivos CSV, alcançando aprovação em 17 dos 17 casos de teste automatizados (100% de êxito). Adicionalmente, o aprimoramento da interface gráfica Swing com laços de validação reativa (`pedirTexto` e `pedirInteiro`) eliminou o cancelamento acidental de fluxos de entrada por erros de digitação do usuário.

Como trabalhos futuros, a elasticidade proporcionada pela classe associativa `Matricula` e pelo desacoplamento em camadas permite expandir o sistema sem impacto estrutural nas classes existentes, destacando-se: (a) o acréscimo de atributos de desempenho acadêmico (notas parciais, média final e percentual de frequência) e do estado `APROVADO`/`REPROVADO` diretamente em `Matricula`; (b) a modelagem de pré-requisitos entre disciplinas por meio de autoassociação em `Disciplina`; e (c) a criação de uma implementação alternativa de repositório baseada em banco de dados relacional (JDBC), preservando intocadas as camadas `model`, `service` e `view`.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6022**: informação e documentação: artigo em publicação periódica técnica e/ou científica: apresentação. 2. ed. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023**: informação e documentação: referências: elaboração. 2. ed. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 10520**: informação e documentação: citações em documentos: apresentação. 2. ed. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. 3. ed. Rio de Janeiro: ABNT, 2011.

BLOCH, Joshua. **Effective Java**. 3. ed. Boston: Addison-Wesley Professional, 2018.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML**: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2006.

DEITEL, Paul; DEITEL, Harvey. **Java**: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2017.

EVANS, Eric. **Domain-driven design**: tackling complexity in the heart of software. Boston: Addison-Wesley Professional, 2003.

FOWLER, Martin. **Patterns of enterprise application architecture**. Boston: Addison-Wesley Professional, 2002.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Padrões de projetos**: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA. **Normas de apresentação tabular**. 3. ed. Rio de Janeiro: IBGE, 1993.

LARMAN, Craig. **Utilizando UML e padrões**: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo. 3. ed. Porto Alegre: Bookman, 2007.

MARTIN, Robert Cecil. **Arquitetura limpa**: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

ORACLE. **Java Platform, Standard Edition & Java Development Kit Version 17 API Specification**. Redwood Shores: Oracle Corporation, 2021. Disponível em: https://docs.oracle.com/en/java/javase/17/docs/api/. Acesso em: 15 out. 2026.